

Section 1: Theoretical Analysis

Q1.1 GEMM Operational Intensity

Q1.1.1. Formula derivation

For a GEMM in one transformer linear layer,

$$Y = XW$$

where

$$X \in \mathbb{R}^{M \times K}, \quad W \in \mathbb{R}^{K \times N}, \quad Y \in \mathbb{R}^{M \times N}.$$

Each output element of (Y) is a dot product of length (K). Counting one multiply-add as 2 FLOPs, the total compute is

$$\text{FLOPs} = 2MKN.$$

For memory traffic, using BF16 means each element is 2 bytes. Assuming each input and weight element is read once and each output element is written once, the memory traffic is

$$\text{Bytes} = 2(MK + KN + MN).$$

Therefore, the operational intensity is

$$\text{OI}_{\text{GEMM}} = \frac{\text{FLOPs}}{\text{Bytes}} = \frac{2MKN}{2(MK + KN + MN)} = \frac{MKN}{MK + KN + MN} \quad \text{FLOP/byte.}$$

This formula shows why larger (M) improves operational intensity: the weight matrix (W) can be reused across more rows of (X), so the amount of compute grows faster than the amount of memory read for the weights.

Q1.1.2. Operational intensity table

Operation	K	N	M=1	M=4	M=16	M=64
Q projection	768	768	1.00	3.96	15.36	54.86
K projection	768	768	1.00	3.96	15.36	54.86
V projection	768	768	1.00	3.96	15.36	54.86
Output projection	768	768	1.00	3.96	15.36	54.86
MLP fc1 (up)	768	3072	1.00	3.97	15.59	57.96
MLP fc2 (down)	3072	768	1.00	3.97	15.59	57.96

Operation	K	N	M=128	M=256	M=512	M=1024
Q projection	768	768	96.00	153.60	219.43	279.27
K projection	768	768	96.00	153.60	219.43	279.27
V projection	768	768	96.00	153.60	219.43	279.27
Output projection	768	768	96.00	153.60	219.43	279.27
MLP fc1 (up)	768	3072	105.93	180.71	279.27	384.00
MLP fc2 (down)	3072	768	105.93	180.71	279.27	384.00

Q1.1.3. Compute-bound vs memory-bound

The TPU v5e ridge point is approximately (240) FLOP/byte. Therefore, entries below (240) FLOP/byte are memory-bound, and entries above (240) FLOP/byte are compute-bound.

For Q, K, V, and output projection, (M=1,4,16,64,128,256,512) are memory-bound, and (M=1024) is compute-bound. The transition occurs between (M=512) and (M=1024).

For MLP fc1 and MLP fc2, (M=1,4,16,64,128,256) are memory-bound, and (M=512,1024) are compute-bound. The transition occurs between (M=256) and (M=512).

Q1.2 Attention Operational Intensity

Q1.2.1. Prefill attention

For batch size (1), prompt length (p), number of heads (H), and head dimension (D):

$$Q, K, V, O \in \mathbb{R}^{1 \times H \times p \times D},$$

For each head:

$$QK^T : (p \times D)(D \times p) \rightarrow p \times p, \quad \text{FLOPs} = 2p^2D.$$

$$\text{Attn}V : (p \times p)(p \times D) \rightarrow p \times D, \quad \text{FLOPs} = 2p^2D.$$

$$\text{FLOPs}_{\text{prefill}} = H \cdot (2p^2D + 2p^2D) = 4Hp^2D.$$

$$\text{Bytes}_{\text{prefill}} = 2 \cdot (|Q| + |K| + |V| + |O|) = 2 \cdot (4HpD) = 8HpD.$$

$$\text{OI}_{\text{prefill}} = \frac{4Hp^2D}{8HpD} = \frac{p}{2} \text{ FLOP/byte.}$$

Q1.2.2. Decode attention

For batch size (1), context length (c), number of heads (H), and head dimension (D):

$$Q \in \mathbb{R}^{1 \times H \times 1 \times D}, \quad K, V \in \mathbb{R}^{1 \times H \times c \times D}, \quad O \in \mathbb{R}^{1 \times H \times 1 \times D}.$$

For each head:

$$QK^T : (1 \times D)(D \times c) \rightarrow 1 \times c, \quad \text{FLOPs} = 2cD.$$

$$\text{Attn}V : (1 \times c)(c \times D) \rightarrow 1 \times D, \quad \text{FLOPs} = 2cD.$$

$$\text{FLOPs}_{\text{decode}} = H \cdot (2cD + 2cD) = 4HcD.$$

$$\text{Bytes}_{\text{decode}} = 2 \cdot (|Q| + |K| + |V| + |O|) = 2 \cdot (HD + HcD + HcD + HD) = 4HD(c + 1).$$

$$\text{OI}_{\text{decode}} = \frac{4HcD}{4HD(c + 1)} = \frac{c}{c + 1} \text{ FLOP/byte.}$$

Q1.2.3. Numerical OI table

For both `small` and `default`, (H) and (D) cancel out:

$$\text{OI}_{\text{prefill}} = \frac{p}{2}, \quad \text{OI}_{\text{decode}} = \frac{c}{c + 1}.$$

Ridge point:

240 FLOP/byte.

(p) or (c)	Prefill OI (=p/2)	Prefill bound	Decode OI (=c/(c+1))	Decode bound
128	64	Memory-bound	0.992	Memory-bound
256	128	Memory-bound	0.996	Memory-bound
512	256	Compute-bound	0.998	Memory-bound
1024	512	Compute-bound	0.999	Memory-bound
2048	1024	Compute-bound	1.000	Memory-bound
4096	2048	Compute-bound	1.000	Memory-bound
8192	4096	Compute-bound	1.000	Memory-bound

Q1.3 Prefill vs Decode

Q1.3.1. Why prefill is compute-bound

For prefill, $\text{OI} = p/2$. As prompt length p increases, operational intensity grows linearly. Structurally, prefill processes all prompt tokens in parallel: QK^T and $\text{Attn}V$ scale as $O(p^2)$, while reading Q , K , V and writing O scales as $O(p)$. Therefore long-prefill attention has high arithmetic intensity and becomes compute-bound.

Q1.3.2. Why decode is memory-bound

For decode attention, $\text{OI} = c/(c+1) \approx 1$. This is far below the TPU v5e ridge point of 240 FLOP/byte, so decode remains memory-bound. Structurally, autoregressive decoding processes only one new query token at a time but must read the entire K/V cache of length c . Both compute and memory traffic scale as $O(c)$, so arithmetic intensity stays low.

Q1.3.3. Hardware utilization implication

Prefill has high arithmetic intensity, so the relevant utilization metric is compute throughput:

Prefill metric: TFLOP/s.

Decode has low arithmetic intensity and repeatedly reads the K/V cache, so the relevant utilization metric is memory bandwidth:

Decode metric: GB/s.

Section 2: Microbenchmarking

Q2.1 Prefill Attention Performance

Q2.1.1. Vary sequence length

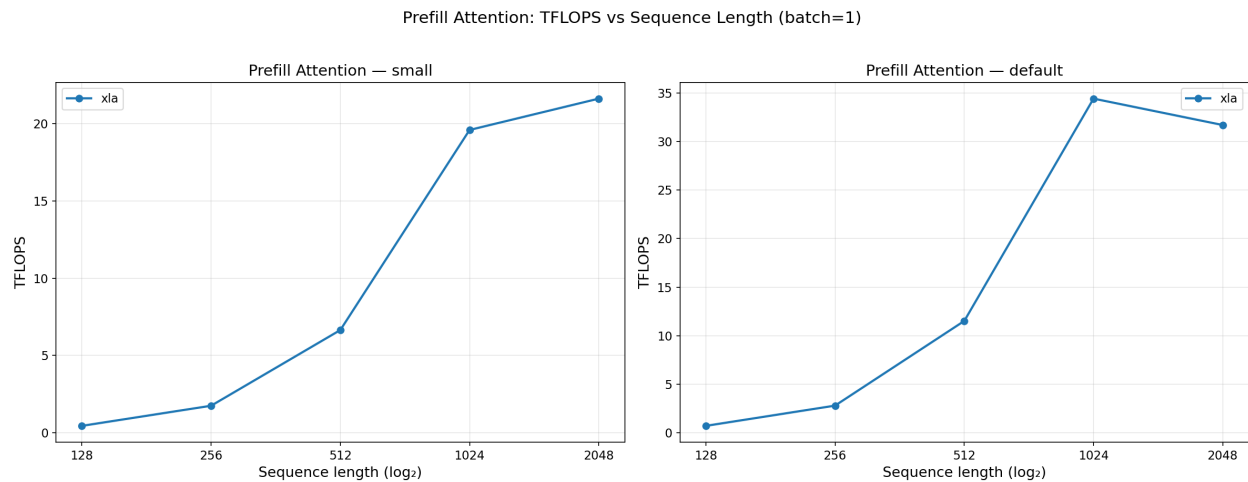


Figure 1: Prefill attention TFLOPS vs sequence length

TFLOPS increases as sequence length grows because prefill attention has $OI=p/2$, so longer prompts have higher arithmetic intensity.

Q2.1.2. Vary batch size

Increasing batch size also generally improves TFLOPS by increasing parallel work and utilization.

Q2.1.3. Scaling and saturation

The sequence-length sweep begins to level off around $p=1024$ to $p=2048$. The batch-size sweep is less monotonic: batch size 4 shows a dip, likely due to shape-specific XLA/TPU kernel efficiency or measurement noise. Overall, larger sequences and larger batches expose more parallel work to the TPU and improve compute utilization.

Prefill Attention: TFLOPS vs Batch Size (seq_len=1024)

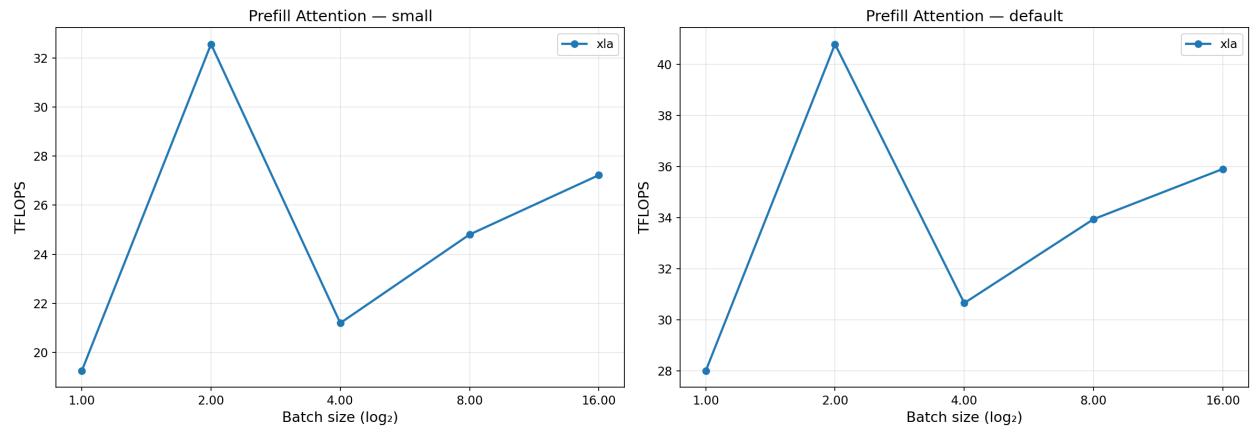


Figure 2: Prefill attention TFLOPS vs batch size

Decode Attention: Bandwidth Utilization vs Context Length (batch=1)

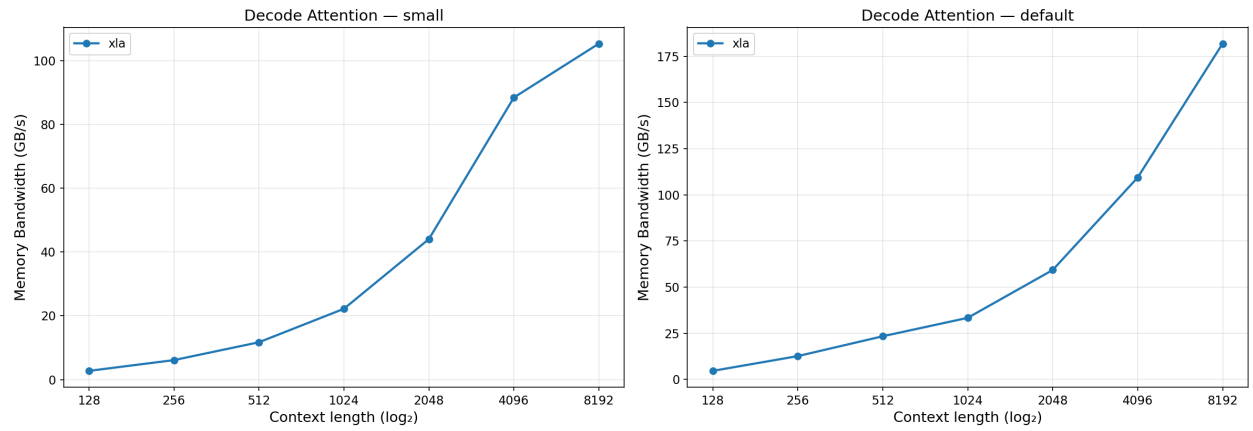


Figure 3: Decode attention bandwidth vs context length

Q2.2 Decode Attention Performance

Q2.2.1. Vary context length

Memory bandwidth increases as context length grows because each decode step reads a larger K/V cache.

Q2.2.2. Vary batch size

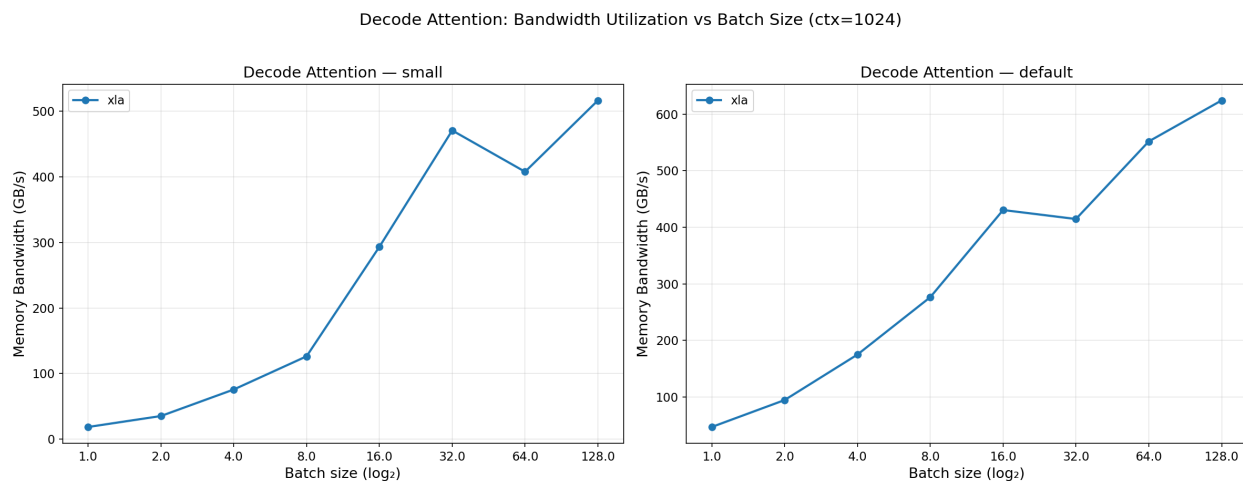


Figure 4: Decode attention bandwidth vs batch size

Increasing batch size generally improves bandwidth utilization because more requests read from the K/V cache in parallel.

Q2.2.3. Bandwidth metric and peak utilization

Decode attention uses memory bandwidth as the main metric because

$$OI_{\text{decode}} = \frac{c}{c + 1} \approx 1,$$

which is far below the TPU v5e ridge point of 240 FLOP/byte. Thus decode is memory-bound rather than compute-bound.

The measured bandwidth reaches roughly 500 GB/s for the small model and 620 GB/s for the default model, compared with the TPU v5e peak of 819 GB/s. This is about 61% and 76% of peak bandwidth, respectively. The remaining gap comes from kernel overhead, non-ideal memory access patterns, XLA/runtime overhead, and shape-specific inefficiencies.

Section 3: End-to-End Inference

Q3.1 Prefill Latency Analysis

Q3.1.1. Prefill latency vs prompt length

Prefill latency increases with prompt length. This is expected because the full-model prefill path includes all transformer-layer work, and attention contains (QK[⊤]) and (AttnV) terms that grow with prompt length. Throughput initially improves as longer prompts increase TPU utilization, but at very long prompts the (O(p²)) attention cost dominates and throughput drops.

Q3.1: Prefill Performance vs Prompt Length

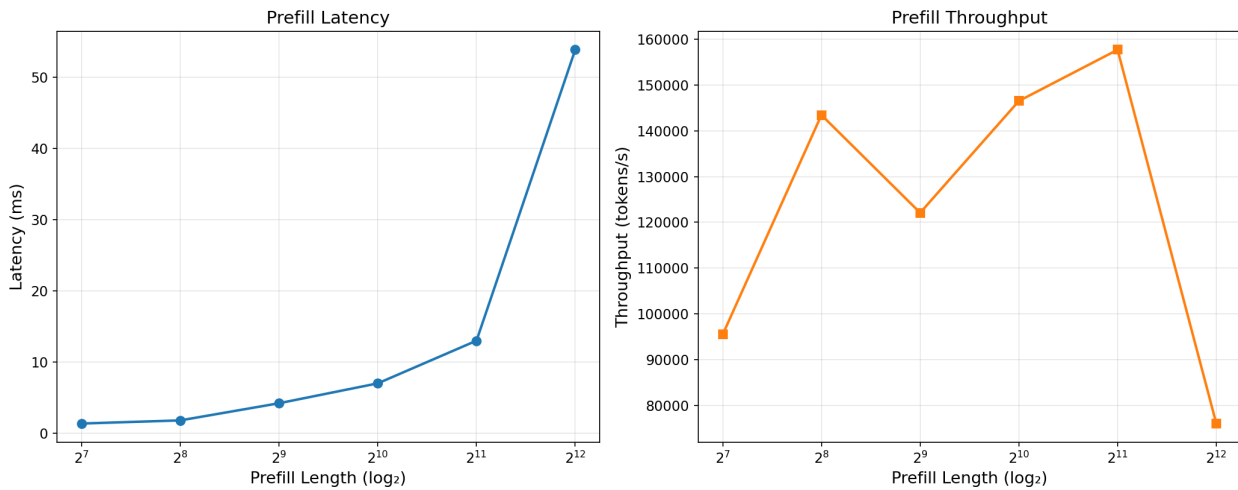


Figure 5: End-to-end prefill latency

Q3.1.2. Profiler traces

Trace Viewer screenshot for short prefill, (p=128):

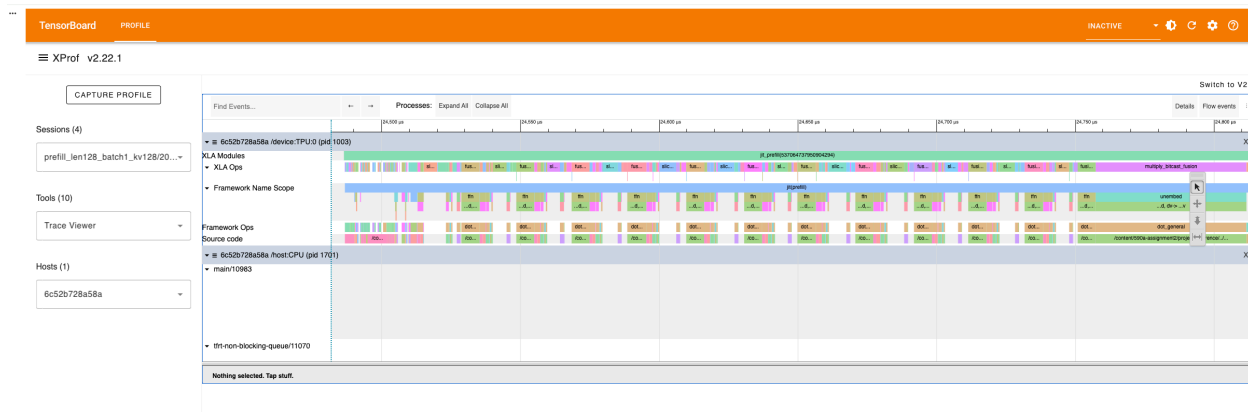


Figure 6: Trace Viewer for prefill length 128

Trace Viewer screenshot for long prefill, (p=4096):

Q3.1.3. Operation breakdown

The Trace Viewer shows repeated transformer-layer regions containing:

embedding $\rightarrow$ QKV projection $\rightarrow$ attention $\rightarrow$ output projection $\rightarrow$ MLP $\rightarrow$ norm/unembed.

The main compute-heavy regions are the projection GEMMs, MLP GEMMs, and attention. Embedding, normalization, and small fusion kernels are visible but take a smaller fraction of the prefill timeline.

Attention detail in the long-prefill trace:

Q3.1.4. Short vs long prefill

For short prefill length ($p=128$), the fixed per-layer work such as QKV projection, output projection, MLP, embedding, and normalization is relatively more visible. Attention is present, but it is not overwhelmingly dominant.

For long prefill length ($p=4096$), attention becomes much more important because prefill attention scales as

$$\text{compute} \sim O(p^2), \quad OI_{\text{prefill}} = \frac{p}{2}.$$

This matches the Section 1 analysis: long prefill has higher arithmetic intensity and is more compute-bound.

Q3.2 Decode Latency and Throughput

Q3.2.1. Decode length sweep

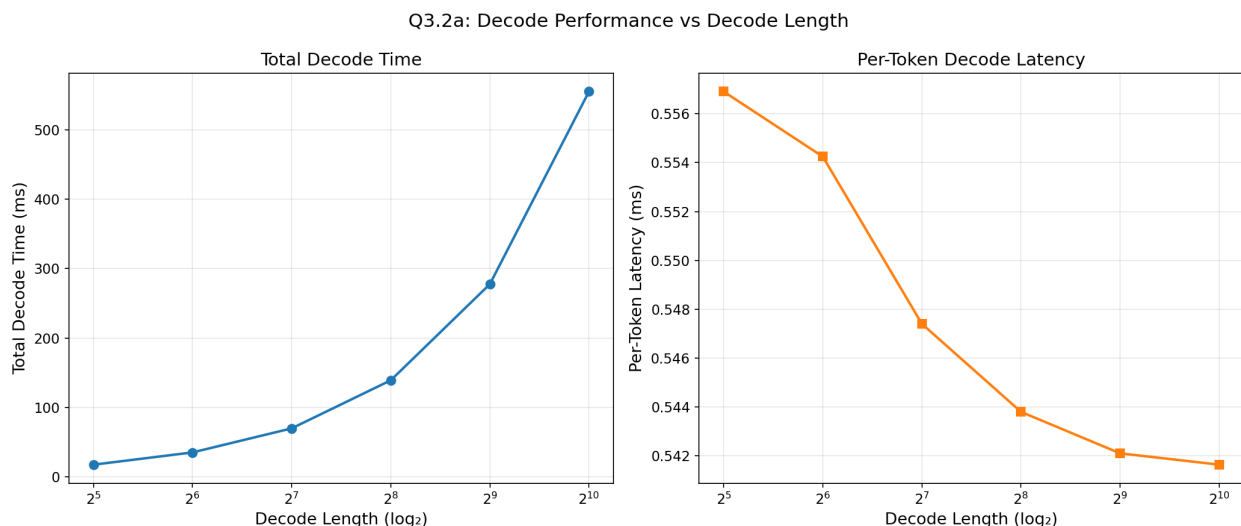


Figure 9: End-to-end decode latency

Total decode time increases with decode length d . Per-token decode latency stays roughly constant, around 0.54 to 0.56 ms/token.

Each decode step generates one new token and reads the current K/V cache. The cache grows during generation, so per-token latency can increase slightly, but in this measurement range the change is small.

Q3.2.2. Batch-size throughput sweep

Throughput increases as batch size grows, then begins to saturate around batch size 16 to 32.

Batching improves decode throughput because decode is memory-bound:

$$OI_{\text{decode}} \approx 1.$$

Larger batches expose more parallel requests and improve memory-bandwidth utilization when reading the K/V cache. Once bandwidth and kernel utilization approach their limits, throughput gains become smaller.

Q3.2b: End-to-End Performance vs Batch Size

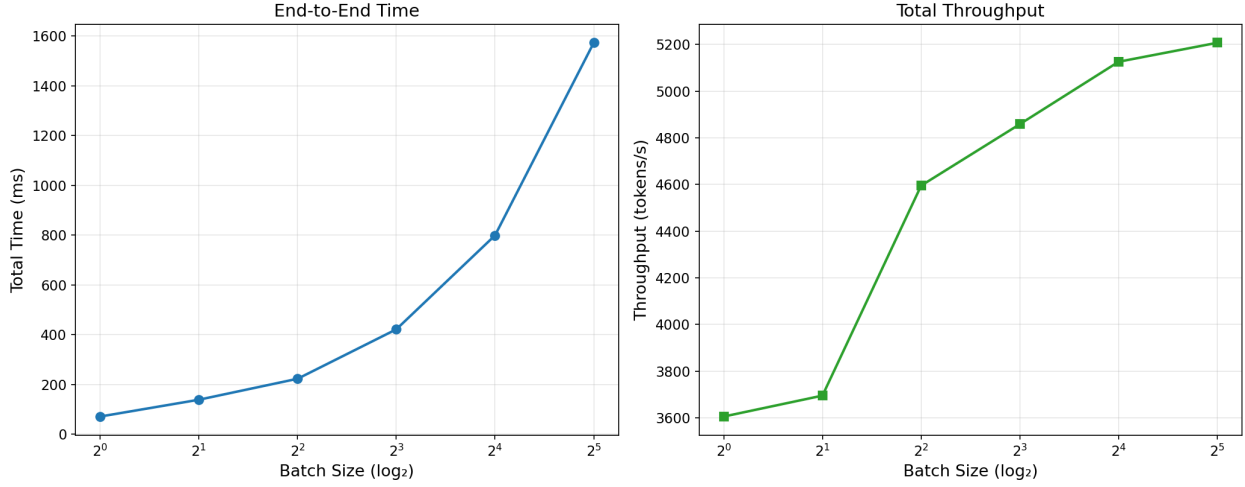


Figure 10: End-to-end batch throughput

Q3.3 Memory Footprint

Q3.3.1. Model weights and K/V cache

For the `small` model:

$$d_{\text{emb}} = 768, \quad L = 12, \quad H_{kv} = 8, \quad D = 96, \quad V = 50304.$$

Model parameters:

$$\begin{aligned} \text{embedding} &= V d_{\text{emb}} = 50304 \cdot 768 = 38,633,472, \\ \text{lm head} &= 768 \cdot 50304 = 38,633,472, \\ \text{per-layer attention} &= 4 \cdot 768 \cdot 8 \cdot 96 = 2,359,296, \\ \text{per-layer MLP} &= 768 \cdot 3072 + 3072 \cdot 768 = 4,718,592. \end{aligned}$$

$$\text{total params} = 38,633,472 + 38,633,472 + 12(2,359,296 + 4,718,592) = 162,201,600.$$

BF16 uses 2 bytes per parameter:

$$\text{weight memory} = 162,201,600 \cdot 2 = 324,403,200 \text{ bytes} \approx 309.4 \text{ MiB}.$$

For K/V cache with sequence length ($S=1024$):

$$\text{KV bytes} = B \cdot S \cdot L \cdot H_{kv} \cdot D \cdot 2_{K,V} \cdot 2_{\text{BF16}}.$$

$$\text{KV bytes per batch item} = 1 \cdot 1024 \cdot 12 \cdot 8 \cdot 96 \cdot 2 \cdot 2 = 37,748,736 \text{ bytes} = 36 \text{ MiB}.$$

Batch size (B)	KV cache memory
1	36 MiB

Batch size (B)	KV cache memory
4	144 MiB
8	288 MiB
16	576 MiB

The K/V cache exceeds model weight memory when

$$B \cdot 36 \text{ MiB} > 309.4 \text{ MiB} \Rightarrow B > 8.59.$$

So the first integer batch size where K/V cache memory exceeds model weight memory is (B=9).

Q3.3.2. Validation

The profiler runs from `bench_e2e.py --config configs/small.yaml --profile validate` the expected trend: increasing batch size increases K/V cache memory linearly. Measured memory can be higher than the theoretical table because XLA also allocates temporary buffers, padded tensors, logits, compiled-program workspace, and other runtime buffers.

Q3.3.3. Maximum batch size

Using only model weights and K/V cache, a 16 GiB HBM upper bound gives

$$B_{\max} = \left\lfloor \frac{16 \cdot 1024 \text{ MiB} - 309.4 \text{ MiB}}{36 \text{ MiB}} \right\rfloor = 446.$$

This is only a theoretical upper bound. In practice, the maximum safe serving batch size is lower because inference also needs memory for XLA temporary buffers, activations, logits, padding, and runtime overhead. The implication is that long-context serving is often constrained by K/V cache memory, so batching improves throughput but eventually becomes limited by HBM capacity.

Section 4: Static and Continuous Batching

Q4.1 Static vs continuous batching

With bimodal decode lengths and `decode_steps=64`:

Policy	Throughput
Static batching	2.31 req/s
Continuous batching	3.90 req/s

Continuous batching has higher throughput and lower end-to-end latency. In the bimodal setting, static batching waits for the longest request in each batch, so short requests waste decode slots. Continuous batching retires completed requests earlier and admits new requests into freed slots.

Q4.2 Effect of `decode_steps`

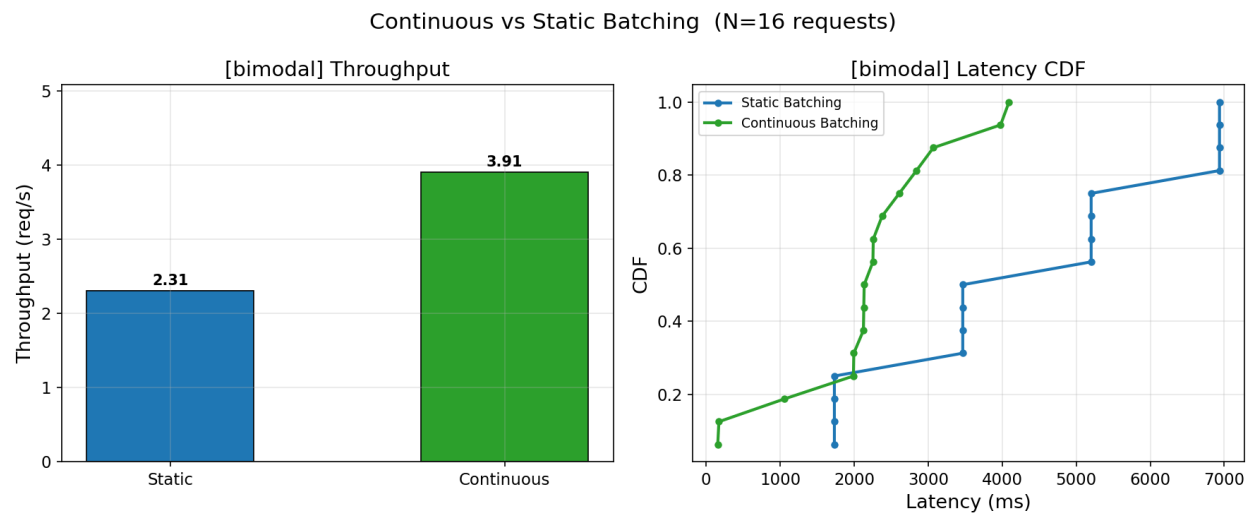


Figure 11: Continuous vs static batching, bimodal distribution

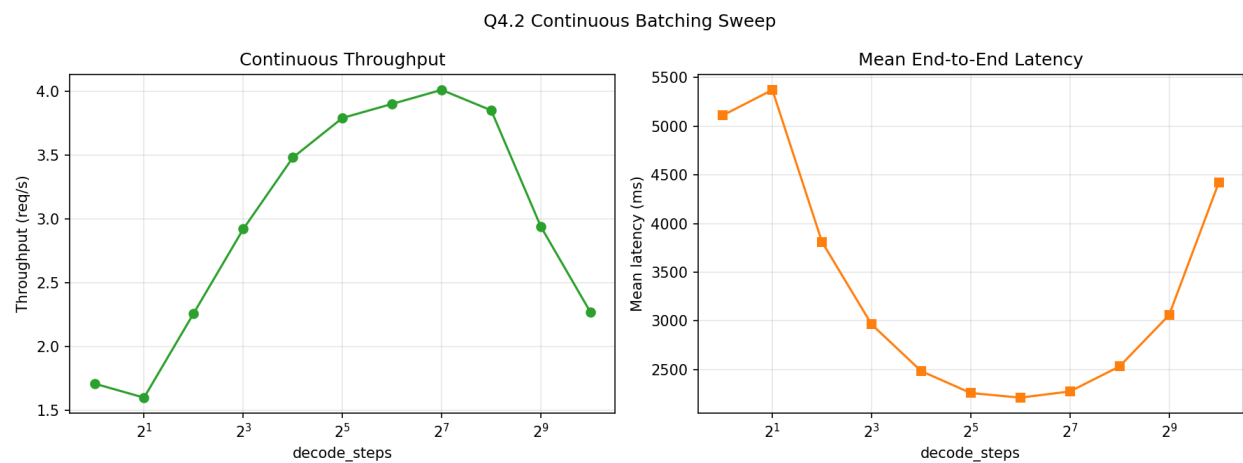


Figure 12: Continuous batching decode steps sweep

decode_steps	Throughput (req/s)	Mean latency (ms)
1	1.71	5111.9
2	1.60	5373.4
4	2.26	3812.2
8	2.92	2964.4
16	3.48	2484.7
32	3.79	2259.1
64	3.90	2211.5
128	4.01	2275.4
256	3.85	2532.2
512	2.94	3060.6
1024	2.27	4419.4

decode_steps controls how many tokens each active request generates before the scheduler checks for completed requests.

Small decode_steps gives fine-grained scheduling, but scheduler and kernel overhead dominate. Very large decode_steps reduces overhead, but short requests cannot retire promptly, so continuous batching becomes static-like. The best region in this experiment is around decode_steps=32 to decode_steps=128.

Q4.3 Fixed request lengths

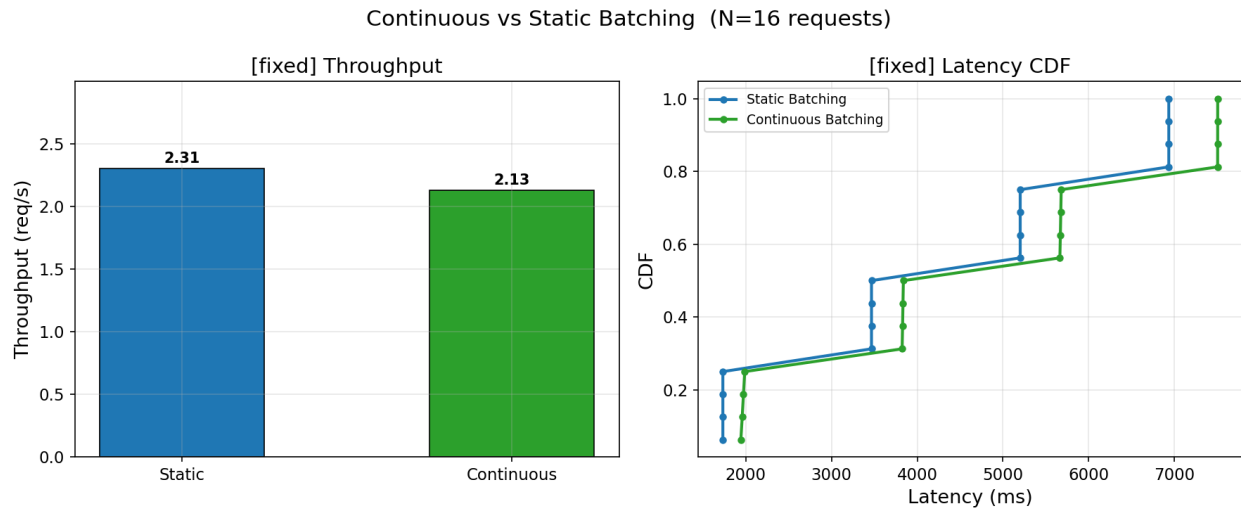


Figure 13: Continuous vs static batching, fixed distribution

With fixed decode lengths:

Policy	Throughput
Static batching	2.31 req/s
Continuous batching	2.13 req/s

When all requests have the same length, continuous batching loses its main advantage: there are no short requests to retire early. Static batching no longer wastes much work waiting for long requests, while continuous batching still pays scheduler and slot-management overhead. Thus fixed-length requests make static batching slightly better in this run.

Section 5: Chunked Prefill

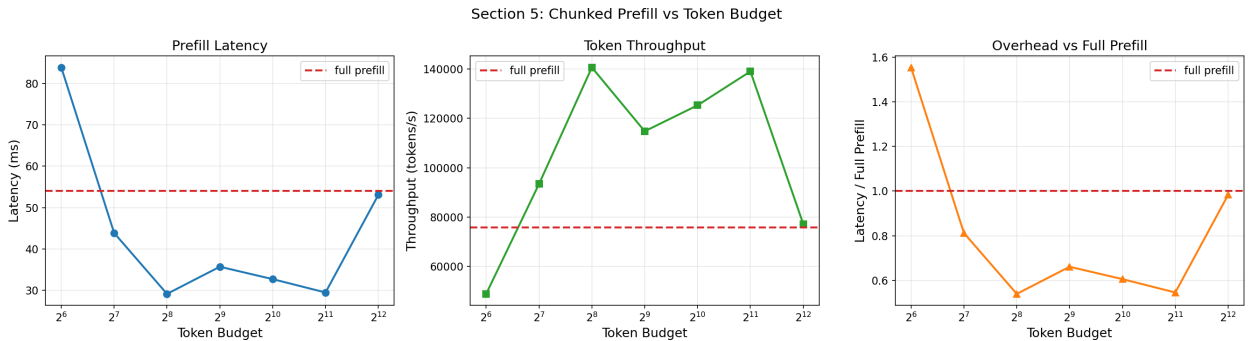


Figure 14: Chunked prefill token-budget sweep

I implemented chunked prefill in `project_inference/bench_chunked_prefill_sol.py`. Instead of pre-filling the full prompt at once, the prompt is split into chunks of size `token_budget`, and each chunk updates the same K/V cache.

Experiment setting:

prefill length = 4096, batch size = 1.

token_budget	Chunks	Latency (ms)	Throughput (tok/s)	Latency / full
64	64	83.89	48,826	1.55
128	32	43.85	93,411	0.81
256	16	29.11	140,698	0.54
512	8	35.68	114,800	0.66
1024	4	32.68	125,324	0.61
2048	2	29.45	139,071	0.55
4096	1	53.07	77,180	0.98

Correctness check:

$$\max |\text{logits}_{\text{chunked}} - \text{logits}_{\text{full}}| = 0, \quad \text{argmax matches full prefill.}$$

Small `token_budget` values create many chunks, so launch/scheduling overhead dominates. Medium budgets, especially 256 to 2048, give the best latency and throughput in this run. At `token_budget=4096`, chunked prefill becomes equivalent to full prefill.